



The Role of Entropy in Guiding a Connection Prover

Zsolt Zombori^{1,2(✉)}, Josef Urban³, and Miroslav Olšák⁴

¹ Alfréd Rényi Institute of Mathematics, Budapest, Hungary
zombori@renyi.hu

² Eötvös Loránd University, Budapest, Hungary

³ Czech Technical University in Prague, Prague, Czechia

⁴ University of Innsbruck, Innsbruck, Austria

Abstract. In this work we study how to learn good algorithms for selecting reasoning steps in theorem proving. We explore this in the connection tableau calculus implemented by `leanCoP` where the partial tableau provides a clean and compact notion of a *state* to which a limited number of inferences can be applied. We start by incorporating a state-of-the-art learning algorithm — a graph neural network (GNN) — into the `plCoP` theorem prover. Then we use it to observe the system’s behavior in a reinforcement learning setting, i.e., when learning inference guidance from successful Monte-Carlo tree searches on many problems. Despite its better pattern matching capability, the GNN initially performs worse than a simpler previously used learning algorithm. We observe that the simpler algorithm is less confident, i.e., its recommendations have higher entropy. This leads us to explore how the entropy of the inference selection implemented via the neural network influences the proof search. This is related to research in human decision-making under uncertainty, and in particular the *probability matching* theory. Our main result shows that a proper entropy regularization, i.e., training the GNN not to be overconfident, greatly improves `plCoP`’s performance on a large mathematical corpus.

Keywords: Automated theorem proving · Machine learning · Reinforcement learning · Graph neural networks · Connection calculus · Entropy regularization

1 Introduction

Automated Theorem Proving (ATP) and Interactive Theorem Proving (ITP) are today increasingly benefiting from combinations with Machine Learning (ML) methods [44]. A number of learning-based inference guiding methods have been developed recently, starting with the `leanCoP` [33, 34] style connection tableaux setting [12, 23, 25, 32, 35, 46, 51], later expanding into the E prover’s [38, 39] and Vampire’s [28] superposition setting [9, 18, 19, 29, 42], and HOL’s [15, 16, 41], Coq’s [10] and Isabelle’s [48] tactical settings [4, 6, 13, 14, 17, 31, 50].

The connection tableau calculus as implemented by `leanCoP` is a very good framework for studying combinations of ML and ATP methods [5]. `leanCoP` has a compact Prolog implementation that is both easy to modify and surprisingly efficient. At the same time, unlike in the superposition and tactical setting, the partial tableau provides a clean and compact notion of a *state* to which a limited number of inferences (*actions*) can be applied. This has recently allowed the first experiments with AlphaZero [40] style Monte-Carlo tree search (MCTS) [12] and Reinforcement Learning (RL) [43] of theorem proving in the `rlCoP` [25], `graphCoP` [32] and `plCoP` [51] systems.

In this work, we start by extending `plCoP` with a state-of-the-art learning algorithm – a graph neural network (GNN) [32] – which was designed for processing logical formulae and exhibits several useful invariance properties, namely invariance under renaming of symbols, negation and reordering of clauses and literals. Despite its better pattern matching capability, the GNN initially performs worse than a simpler previously used learning algorithm based on gradient boosted trees (XGBoost [8]). We observe that the simpler algorithm is less confident about the inferences that should be applied to the proof states, i.e., its recommendations have higher entropy, leading to greater exploration of different inferences.

This leads us to analyze how the entropy of the inference selection implemented via the neural network influences the proof search. We try increasingly high penalties for overconfidence (low entropy) during the training of the GNN, using an approach called Maximum Entropy Reinforcement Learning [49]. For this, we need to be able to compare the entropy of proof states with different numbers of possible inferences (actions). We do that by introducing *normalized entropy*, which allows for comparing discrete distributions of different lengths. We make a rather surprising discovery that replacing the particular trained predictors by arbitrary (random) but entropy-normalized predictors that respect the inference ordering yields only slightly weaker ATP performance. This suggests that the correct ordering of possible inferences according to their utility in a given state plus the right amount of entropy capture most of the benefits of the learned guidance. In summary, our contributions are:

1. We integrate a fast logic-aware graph neural network into the `plCoP` system, allowing its use for guiding the choice of inferences (policy) and for estimating the provability of a partial connection tableau (value).
2. We adapt the graph construction algorithm to support the paramodulation inferences used in `plCoP`.
3. We analyze the entropy of the policy and its role in `plCoP`'s performance.
4. We show that random policies with the right ordering and normalized entropy perform already quite well.
5. We do several smaller and larger evaluations over the standard corpus extracted from the Mizar Mathematical Library (MML) and show that the best entropy regularized GNN greatly improves over other learning-guided connection tableaux systems. In particular, we report 17.4% improvement on the Mizar40 evaluation set over `rlCoP`, the best previously published result.

The rest of the paper is structured as follows. Section 2 introduces in more detail the necessary background such as neural guidance of provers, the `leanCoP` setting, reinforcement learning and Monte-Carlo tree search, and Maximum Entropy learning. Section 3 discusses in more depth the use of Maximum Entropy learning in guiding MCTS-based theorem proving. Section 4 describes our new implementation and Sect. 5 experimentally evaluates the methods.

2 Background and Related Work

2.1 Neural Feature Extraction for Guiding Theorem Provers

Learning based ATP systems have for a long time explored suitable characterizations of mathematical objects, leading to solutions that process text directly (e.g. [2, 4, 29]) and solutions that rely on manually engineered features (e.g. [18, 26]). Graph neural networks (GNN) [37] provide an alternative to both approaches: the graph representation allows for retaining the syntactic structure of mathematical objects, while also allowing for end-to-end (i.e., involving no manually designed features) training. However, improving over learning based on manual feature extraction has proven to be challenging with GNNs, especially in real time, as noted in several works (e.g. [9, 11]). Usually it required high level of technical engineering. The GNN presented in [32] was designed to preserve many useful invariance properties of logical formulae and has demonstrated impressive improvement in guiding the `leanCoP` connection calculus compared with gradient boosted trees. We refer to this system as `graphCoP`.

2.2 Systems Guiding the `leanCoP` Theorem Prover

`leanCoP` [34] is a compact theorem prover for first-order logic, implementing connection tableau search. The proof search starts with a *start clause* as a *goal* and proceeds by building a connection tableau by applying *extension steps* and *reduction steps*. `leanCoP` uses iterative deepening to ensure completeness.

A series of learning systems guiding the `leanCoP` connection calculus have been developed recently. Of these, we highlight three that use roughly the same reinforcement learning setup: `rlCoP` [25], `plCoP` [51] and `graphCoP` [32]. These systems search for proofs using Monte Carlo Tree Search [7] and they train the *value* (the proof state quality) and the *policy* (the inference quality in a proof state) functions similarly to systems like AlphaZero [3, 40]. `rlCoP` and `plCoP` use manually developed features [26] and gradient boosted trees (XGBoost [8]) for learning while `graphCoP` employs a GNN for end-to-end feature extraction and learning. This graph neural network was designed for processing mathematical formulae and has several useful invariance properties: the graph structure is invariant under renaming of symbols, negation and reordering of clauses and literals. In this paper, we incorporate the GNN of [32] into `plCoP`.

The `plCoP` system extends `leanCoP` with paramodulation steps that can handle equality predicates more efficiently. Let $t|_p$ denote the subterm of t at position

p and $t[u]_p$ denote the term obtained after replacing in t at position p by term u . Given a goal G and an input clause¹ $\{X \neq Y, B\}$, such that, for some position p there is a substitution σ such that $G|_p\sigma = X\sigma$, the paramodulation step changes G to $\{G[Y]_p\sigma, B\sigma\}$. Rewriting is allowed in both directions, i.e., the roles of X and Y can be switched.

2.3 Reinforcement Learning (RL)

Reinforcement learning (RL) [43] aims to find the optimal behaviour in an environment defined as a Markov Decision Process (MDP). An MDP($\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma$) describes a dynamic process and consists of the following components: $\mathcal{S}$ is the set of possible states, $\mathcal{A}$ is the set of possible actions, $\mathcal{R} : (\mathcal{S} \times \mathcal{A}) \rightarrow \mathbb{R}$ is a reward function, $\mathcal{P} : (\mathcal{S} \times \mathcal{A}) \rightarrow \mathcal{S}$ is the state transition function and γ is the discount factor. We assume that an agent interacts with this MDP, generating sequences of (s_t, a_t, r_t) state-action-reward tuples, called *trajectories*. The agent is equipped with a *policy* function $\pi : \mathcal{S} \rightarrow \mathcal{A}$ which determines which action it selects in a particular state. The policy is often stochastic, i.e., it defines a probability distribution over inferences that are possible in a given state. The aim of the agent is to maximize its total accumulated reward $\sum_{t \geq 0} \gamma^t r_t$. Several components, such as the reward and transition functions, can be stochastic, in which case the aim of the agent is to find the policy π^* that maximizes its cumulative expected reward, where future rewards are discounted with the γ discount factor:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t | \pi \right]$$

2.4 Monte Carlo Tree Search (MCTS)

MCTS is a simple RL algorithm, which builds a search tree whose nodes are states and where edges represent actions. The aim of the search algorithm is to find trajectories (branches in the search tree) that yield high accumulated rewards. The search starts from a single root node, and new nodes are added iteratively. In each node i , we maintain the number of visits n_i , the total reward r_i , and the prior probability (estimated typically by a trained predictor) p_i of all its possible successors (in our case produced by the possible inferences). Each iteration, also called *playout*, involves the addition of a new leaf node.

The policy π used for selecting the actions of the playout is based on the standard UCT [27] (Upper Confidence Trees) formula (1): in each state we select the action with the maximal UCT value in the successor state. Once a new leaf state is created, we observe its reward and update its ancestors: visit counts are increased by 1 and rewards are increased by the reward of the leaf.

$$\text{UCT}(i) = \frac{r_i}{n_i} + cp \cdot p_i \cdot \sqrt{\frac{\ln N}{n_i}} \quad (1)$$

¹ Assuming Disjunctive Normal Form.

In (1), N is the number of visits of the parent, and cp is a parameter that determines the balance between nodes with high reward (exploitation) and rarely visited nodes (exploration). In [3, 40] MCTS is augmented with two learned functions. The *value* function estimates the accumulated reward obtainable from a state, and the leaf nodes are initialized with this value estimates. The second function predicts the prior probability of state-action pairs, which is usually referred to as the *policy*, with a slight abuse of terminology. When it can lead to confusion, we refer to this policy as π^M .

The plCoP, rlCoP and graphCoP systems use the MaLARea/DAGger [36, 45] meta-learning algorithm to learn the policy and value functions. They interleave ATP runs based on the current policy and value (*data collection phase*) with a *training phase*, in which these functions are updated to fit the collected data. Such iterative interleaving of proving and learning has been used successfully in ATP systems such as MaLARea [45] and ENIGMA [20].

During the proof search we build a Monte Carlo tree for each training problem. Its nodes are the proof states (partial tableaux), and the edges represent inferences. Note that the Monte Carlo tree is thus different from the tableau trees. A branch of this Monte Carlo tree leading to a node with a closed tableau is a valid proof. Initially, the three leanCoP-based systems use somewhat different heuristic value and policy functions, later to be replaced with the learned guidance. To enforce deeper exploration, we perform a *bigstep* after a fixed number of playouts: the starting node of exploration is moved one level down towards the child with the highest value (called the *bigstep node*). Later MCTS steps thus only extend the subtree under the bigstep node. This in practice means no backtracking of the bigsteps, which in turn involves giving up completeness.

2.5 Maximum Entropy Reinforcement Learning

When training the policy, directly maximizing the expected utility on the action sequences observed by an RL agent (i.e., the training examples) can lead to instability. The policy can get stuck in local minima and become overconfident, preventing it from exploring the search space sufficiently when necessary to make good decisions. This has motivated using stochastic policies and several *regularization* (i.e., encouraging generality) techniques that ensure that all actions have a chance of being selected. Another motivation for properly regularized stochastic policy learning comes from experiments on humans and animals, suggesting that biological agents do not deterministically select the action with the greatest expected utility [47]: instead they randomly select actions with probability proportional to the expected utility, called *probability matching*. Consequently, action sequences that generate similar rewards tend to be similarly probable, i.e., we avoid making strong commitments whenever it is possible. Maximum Entropy Reinforcement Learning (MaxEnt RL), achieves probability matching by adding an entropy term to the loss function when the policy is trained:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t + \alpha H_{\pi}[a|s_t] \right] \pi$$

where $H_\pi[a|s_t]$ is the Shannon entropy of the probability distribution over valid actions in state s_t :

$$H[p] = - \sum_{i=1}^n p_i \log(p_i)$$

and α is the entropy coefficient. This means that the training of the policy will be maximizing a weighted sum of the (discounted) rewards and of the entropy of the resulting distribution, thus discouraging overconfidence. The entropy term in the objective was first used in [49] and since then its benefit has been empirically demonstrated in several domains. It is particularly useful in dynamic environments, where some uncertainty remains, irrespective of the amount of exploration.

2.6 Kullback-Leibler Divergence

Shannon’s entropy measures the uniformity of a single distribution. However, when contrasting different policies, we will need a measure to compare different distributions. For this, one of the most widely used options is the Kullback-Leibler (KL) divergence, also called *relative entropy*, which is a measure of how one probability distribution differs from a given reference distribution. For a discrete target distribution Q and a reference distribution P , the KL divergence is defined as:

$$KL(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

This measure is zero exactly when P and Q are identical, otherwise it is positive. It can be infinite if there is some x such that $P(x) > 0$ and $Q(x) = 0$. A small $KL(P\|Q)$ means that the two distributions are similar on the domain where most of the probability mass of P lies. Note that $KL(P\|Q) \neq KL(Q\|P)$. For example, consider the following distributions:

$$\begin{aligned} P &= [0.5, 0.47, 0.01, 0.01, 0.01] \\ Q &= [0.96, 0.01, 0.01, 0.01, 0.01] \end{aligned}$$

$KL(P\|Q) = 1.48$ and $KL(Q\|P) = 0.58$. When the summed terms are weighted according to P in $KL(P\|Q)$, the first two terms get large weight, while only the first term gets large weight in $KL(Q\|P)$. When both KL divergences are small, it is a good indicator of similarity of the two distributions.

3 Maximum Entropy for MCTS and Theorem Proving

In this section we discuss the entropy of the inference policy and its potential influence on the proof search. We also introduce a *normalized entropy* to correctly handle probability vectors of different lengths and argue that MaxEnt RL is more targeted and powerful than previously used temperature-based entropy control. Section 4 then describes our implementation.

3.1 Exploration and Entropy in MCTS

The MCTS implemented via the UCT formula has a built-in mechanism for balancing the *exploitation* of proof states that already have high rewards and the *exploration* of inferences whose effect is not yet known. This balancing serves to mitigate errors in the proof state value estimates. However, we need to do another balancing within exploration, between the different under-explored inference branches. This is estimated by the π^M policy predictor as the prior probabilities of the possible inferences. Hence, besides the ordering of the inferences that are possible from a given state, their exact prior probabilities are important as they determine how the exploration budget is split between them. This observation directs our attention to the entropy (uncertainty) of π^M and its relation to the theorem proving performance.

We argue that finding the right level of (un)certainty is particularly important for theorem proving. The goal of learning is to acquire inductive biases that allow the system to perform well on novel problems.² In many situations, however, it is not realistic to extract enough knowledge from the training data that justifies a high level of confidence. Sometimes, there is just not enough analogy between a new problem and the training problems, and we would like our guidance to be more conservative so that we can explore all directions equally. This makes a strong case for using MaxEnt RL, which gives tools for shaping the entropy (uncertainty) profile of our learned π^M policy predictor.

3.2 Normalized Entropy

In this work, we empirically demonstrate the importance of including the “right amount” of entropy when training the policy that guides the theorem prover. To the best of our knowledge, this is the first time that the effect of entropy regularization for MCTS in general and for theorem proving in particular is examined.

Using standard entropy for comparing probability vectors of different length would, however, be misleading. The same entropy value can mean very different uncertainty if the length of the vector changes. For example, consider the vectors

$$\begin{aligned} &[0.34, 0.33, 0.33] \\ &[0.73, 0.07, 0.05, 0.05, 0.05, 0.01, 0.01, 0.01, 0.01, 0.01] \end{aligned}$$

Their entropy is roughly the same (1.1), despite the fact that the first is nearly uniform and the second centers around its first value. To make the uncertainty of these two vectors comparable, we introduce *normalized entropy*:

Definition 1. Given a discrete probability vector p of length n , let $H^*[p] = H[p]/\log(n)$ denote the normalized entropy of p .

² In this sense, theorem proving can be considered as a meta learning task.

Here, $\log(n)$ is the entropy of the uniform distribution when the length is n , hence it is the upper bound of $H[p]$. Consequently, $H^*[p] \in [0, 1]$. Furthermore, it is dimensionless, i.e., it does not depend on the base of the logarithm. The difference between the two distributions in the example above is better captured by their normalized entropy, which is 1 and 0.48.

3.3 Temperature-Based and Regularization-Based Entropy Control

An alternative mechanism for injecting entropy into the policy is through the softmax *temperature* parameter T . Our policy predictors (both XGBoost and GNN) output an unconstrained *logit* vector l , which is normalized to a probability vector p using the softmax function:

$$p_i = \frac{e^{\frac{l_i}{T}}}{\sum_{j=1}^n e^{\frac{l_j}{T}}}$$

Increasing the temperature flattens the probability curve, approaching the uniform distribution in the limit. On the other hand, if the temperature gets close to 0, then most of the probability mass concentrates on the most likely action.

While both higher temperature and entropy regularization increase the ultimate entropy of the policy, they work differently. The temperature acts globally and uniformly, flattening all inference probabilities estimated by the trained policy predictor. Entropy regularization, on the other hand, is part of the training process and it allows the neural network to learn distinguishing between situations with low and high uncertainty. In obvious situations, entropy regularization does not prevent the neural network from acquiring great certainty, while it will drive the network to more uniform predictions when there is no strong evidence against that in the training data. Hence, we expect entropy regularization to be more targeted and powerful than the temperature optimization. This is empirically demonstrated in Sect. 5.

4 Entropy Regularized Neural Guidance for pICoP

This section gives an overview of the training procedure, including how data is extracted from tableaux and Monte Carlo trees.

4.1 Neural Representation of the State and Inference Steps

The proof state in the leanCoP setting is roughly described by the partial tableau and by the set of input clauses corresponding to the initial axioms and conjecture. Each time we choose an extension step, we map the state into a hypergraph, as described in [32]. In more detail, we use the current goal (single literal), the active path leading to the current goal (set of literals), the set of all open goals (set of literals), and the input clauses in the hypergraph construction. The GNN processes the hypergraph and outputs a value prediction for the proof state in

the range $[0, 1]$. It also outputs a probability distribution over all the literals of the axiom clauses that can be used for extension steps, i.e., that can be unified with the negation of the current goal literal.

The above method is used already in **graphCoP**, but the hypergraph construction algorithm used by **graphCoP** was designed to guide only the extension steps. Hence it expects the set of all clauses together with the information that identifies literals within the clauses that unify with the negation of the current goal. We adapt this to paramodulation by temporarily creating a clause for each valid paramodulation step that “simulates” the latter as an extension step. Suppose that the current goal is G and there is an input clause $\{X \neq Y, B\}$, such that, for some position p there is a substitution σ such that $G|_p\sigma = X\sigma$. There is a valid paramodulation step that replaces G with $\{G[Y]_p\sigma, B\sigma\}$. We simulate this step as an extension by adding clause $\{\neg G\sigma, G[Y]_p\sigma, B\sigma\}$ to the input clauses, when constructing the graph.

4.2 Training the Policy and Value Guidance for MCTS

As in **plCoP**, the value and policy estimates are stored in the MCTS nodes and are used for guiding proof search. The training data for learning policy (inference probabilities) and value (state quality) are also handled as in **plCoP**. They are extracted from the tableau states of the bigstep nodes. For each bigstep state, the value target is 1³ if it leads to a proof and 0 otherwise. The policy targets at a particular proof state are the relative frequencies of the possible inferences, i.e., the children in the search tree.

For **graphCoP**, a single GNN was jointly trained to predict both the value and the policy [32]. However, we observed that training separate predictors yields a small improvement, hence we conduct our experiments in this setup. Consider a tableau state s for which we want to learn its target value v and target policy $p_1, \dots, p_n$. Suppose that the partially trained value predictor outputs v' and the policy predictor outputs $p'_1, \dots, p'_n$, then the objectives that we minimize are:

- **value objective:** $(v - v')^2$
- **policy objective:** $-\sum_{i=1}^n p_i \cdot \log(p'_i) - \alpha H[p']$

For more details of the graph construction and neural training we refer to [32]. In summary, we use the same setting as there, except for (i) extending guidance to paramodulation steps, (ii) training separate policy and value GNNs, (iii) increasing the number of neural layers in the GNN from 5 to 10,⁴ and (iv) changing the policy training to encourage policies with higher entropy.

5 Experiments

We first introduce our datasets and other experimental settings (Sect. 5.1). Then we show the impact of entropy regularization (Sect. 5.2) and experimentally com-

³ A discount factor of 0.99 is applied to positive rewards to favor shorter proofs.

⁴ This is motivated by the experiments with the ENIGMA-GNN system [18], where 8–10 layers produce better results than 5 layers.

pare the entropies and other characteristics of the XGBoost and GNN predictors (Sect. 5.3). In Sect. 5.4, we show that random policies with the right ordering and normalized entropy perform already quite well. Section 5.5 then compares temperature-based entropy control with our approach. Finally, Sect. 5.6 evaluates the methods in a train/test scenario on the full Mizar40 dataset using several iterations of proving and learning.

5.1 Datasets and Common Settings

We evaluate our system⁵ using the same datasets as those in [25]. The *Mizar40* dataset [22] consists of 32524 problems from the Mizar Mathematical Library that have been proven by several state-of-the-art ATPs used with many strategies and high time limits in the experiments described in [24]. Based on the proofs, the axioms were ATP-minimized, i.e., only those axioms were kept that were needed in any of the ATP proofs found. The smaller *M2k* dataset [21] consists of 2003 Mizar40 problems that come from related Mizar articles. Finally, we use the bushy (small) problems from the MPTP2078 benchmark [1], which contains just an article-based selection of Mizar problems, regardless of their solvability by a particular ATP system.

Unless otherwise specified, we use the same hyperparameters as described in [51], with the following important exceptions. To allow for faster experiments and put more emphasis on guidance instead of search, we reduce the per problem inference limit from 200000 to 20000 and the bigstep frequency from 2000 to 200. Hence the overall search budget is reduced by a factor of 10. We use a very large CPU time limit (300 s) intended to ensure that proof search terminates after exhausting the inference limit.

5.2 Experiment 1: Influence of Entropy Regularization

In this experiment, we examine the effect of regularizing the entropy of our policy predictor. We produce several variants of the GNN policy predictor which differ in the entropy coefficient α used in its training. Table 1 summarizes our results. We find that the entropy coefficient has a big impact on performance. By the 10th iteration, the best GNN predictor with $\alpha = 0.7$ is 17% better than the unregularized GNN and 5% better than XGBoost. Table 1 also shows the average entropy of the policies generated by the predictor during the proof search. Note that the average entropy of the best predictor in most iterations is reasonably close (often the closest) to the entropy of the XGBoost predictor. This suggests that one key strength of the XGBoost predictor is that it hits the “right” amount of entropy. Matching this entropy in the GNN with adequate regularization allows for matching and even surpassing XGBoost in performance.

⁵ The new extensions described here and the experimental configuration files are publicly available at plCoP’s repository: <https://github.com/zsoltzombori/plcop>.

Table 1. Number of problems solved (Succ) and average policy entropy (Ent) on the M2k dataset. α is the entropy loss term coefficient. Best models are marked with **boldface**, best GNN models are underlined.

Model	α	Iter 1		Iter 2		Iter 4		Iter 6		Iter 8		Iter 10	
		Ent	Succ	Ent	Succ	Ent	Succ	Ent	Succ	Ent	Succ	Ent	Succ
XGB		1.41	790	1.29	956	1.22	1061	1.19	1119	1.17	1147	1.14	1171
GNN	0	0.91	746	0.56	850	0.37	938	0.34	992	0.31	1021	0.32	1050
GNN	0.1	0.86	<u>787</u>	0.6	867	0.43	933	0.37	996	0.37	1031	0.38	1070
GNN	0.2	1.11	769	0.71	878	0.51	976	0.51	1045	0.49	1077	0.46	1114
GNN	0.3	1.05	736	0.8	868	0.7	991	0.73	1071	0.69	1109	0.78	1170
GNN	0.5	1.31	781	1.14	884	1.17	1015	1.13	1085	1.12	1144	1.06	1191
GNN	0.6	1.37	759	1.25	889	1.26	1040	1.21	1098	1.18	1150	1.19	1197
GNN	0.7	1.41	727	1.32	854	1.27	<u>1057</u>	1.22	1132	1.24	1184	1.2	1228
GNN	0.8	1.42	757	1.37	<u>912</u>	1.35	1029	1.32	1079	1.29	1111	1.3	1144
GNN	1.0	1.53	742	1.41	911	1.38	1032	1.35	1102	1.36	1144	1.35	1173
GNN	2.0	1.59	725	1.57	782	1.53	894	1.5	1007	1.5	1047	1.5	1086

5.3 Experiment 2: Relative Entropy on the Same Proof States

Table 1 reveals that there is a reasonable match in average entropy between XGBoost policies and our best GNN policies. Note, however, that this is in general measured on different proof states as the policies themselves determine what proof states the prover explores. To gain a deeper understanding, we create a separate dataset of proof states and compare the different GNNs from Experiment 1 with XGBoost on these proof states using the following four metrics: 1) fraction of proof states where the two predictors have the same most probable inference (Best), 2) fraction of proof states where the two predictors yield the same inference ordering (Order), and the average KL divergence (relative entropy) between the predictors in both directions: 3) $KL(X||G)$ and 4) $KL(G||X)$.⁶ We contrast these metrics with the number of problems solved (Succ) by the corresponding entropy regularized GNN.

We perform this comparison using two datasets. These are the set of states visited by an unguided prover on the 1) M2k dataset and the 2) MPTP2078 bushy benchmark. The first set is part of the training corpus, while the second was never seen by the predictors before. The results can be seen in Table 2.

Changing the entropy coefficient mostly does not change the order of actions, as expected. For the two datasets, the GNN and the XGBoost predictors select the same best inference in around 80% and 58% of the states and yield exactly the same inference ordering in around 40% and 22% of the states. This reveals a significant diversity among the two predictor families, suggesting potential in combining them. We leave this direction for future work.

We find that the same level of entropy regularization ($\alpha = 0.7$) is the best when running both on the familiar (M2k) and the previously unseen

⁶ X and G stand for the probability distributions predicted by XGBoost and GNN, respectively.

Table 2. Comparing the differently entropy-regularized GNN predictors (G) with XGBoost (X) on two fixed sets of proof states generated by running an unguided prover on the M2k and MPTP2078 benchmarks. All predictors were trained on M2k for 10 iterations. α is the entropy regularization coefficient. XGBoost solves 1171 (M2K) and 491 (MPTP2078) problems.

α	M2K					MPTP2078b				
	Succ	Best	Order	$KL(X\ G)$	$KL(G\ X)$	Succ	Best	Order	$KL(X\ G)$	$KL(G\ X)$
0	1050	0.81	0.43	0.52	2.9	230	0.56	0.22	0.97	4.5
0.1	1070	0.8	0.44	0.5	2.37	245	0.58	0.24	0.91	3.83
0.2	1114	0.81	0.42	0.47	1.66	256	0.56	0.24	0.88	2.82
0.3	1170	0.82	0.42	0.36	0.58	276	0.56	0.23	0.61	0.9
0.5	1191	0.82	0.42	0.24	0.28	335	0.59	0.23	0.41	0.43
0.6	1197	0.82	0.4	0.22	0.23	359	0.59	0.23	0.36	0.36
0.7	1228	0.82	0.4	0.22	0.21	399	0.58	0.22	0.34	0.32
0.8	1144	0.81	0.39	0.22	0.21	357	0.58	0.22	0.34	0.31
1.0	1173	0.82	0.4	0.24	0.21	363	0.58	0.22	0.33	0.29
2.0	1086	0.81	0.39	0.34	0.26	362	0.58	0.21	0.37	0.3

(MPTP2078) dataset. This is where the two directional KL divergences (relative entropies) roughly coincide and their sum is roughly minimal. These results make a stronger case for the hypothesis from Experiment 1, that the best GNN performance is obtained when the policy distributions are statistically close to those of the XGBoost predictor.

5.4 Experiment 3: Order and Entropy Are Largely Sufficient

Tables 1 and 2 demonstrate the importance of the entropy of the inference policy in ATP performance. To make this even more apparent, we design an experiment in which we remove a large part of the information contained in the inference policy, only preserving the inference ordering and the normalized entropy. The top two lines of Table 3 show the normalized entropy across iterations of the XGBoost and GNN predictors. Note that it is very stable. We select a target normalized entropy H^* and for each length l we generate a fixed random discrete probability p_l of length l whose normalized entropy is H^* . Finally, we run an MCTS evaluation in which each time our policy predictor emits a probability vector of length l , we replace it with p_l , permuted so that the ordering remains the same as in the original policy. Table 3 shows the ATP performance for the differently normalized entropy targets.

We find that the performance of this predictor is surprisingly good: its performance (1154) is only 1% worse than XGBoost (1171), 10% better than unregularized GNN (1050) and 6% worse than the best GNN (1228). This suggests that the right inference ordering plus the right amount of entropy capture most of the benefits of the learned inference guidance.

Table 3. Normalized entropy of the XGBoost and GNN predictors on M2k (top two rows) and number of problems solved by random policies constrained to have the same action ordering and fixed normalized entropy.

Iteration	0	1	2	3	4	5	6	7	8	9	10
XGBoost H^*	1	0.73	0.71	0.69	0.68	0.67	0.67	0.67	0.66	0.67	0.66
GNN ($\alpha = 0.7$) H^*	1	0.83	0.79	0.8	0.79	0.79	0.78	0.78	0.78	0.8	0.78
GNN $H^* = 0.6$	523	700	782	849	909	956	984	1019	1037	1059	1083
GNN $H^* = 0.7$	523	702	800	856	922	954	995	1040	1077	1110	1129
GNN $H^* = 0.8$	523	693	832	938	1023	1054	1086	1077	1115	1129	1154

5.5 Experiment 4: Temperature vs. Entropy Regularization

As noted in Sect. 3, tuning the softmax temperature is an alternative to entropy regularization. For XGBoost, the temperature was previously optimized to be $T = 2$ and all reported experiments use this number. For the GNN predictors, we used the default $T = 1$. In Table 4, we show how the ATP performance of the GNN changes after it has been trained for 10 iterations on the M2k dataset (without entropy regularization). Increasing the temperature brings some improvement, however, this is much smaller than the benefit of entropy regularization. This is true even if we take the best temperature ($T = 4$) and perform a full 10 iteration training with this temperature, as shown in Table 5. We obtain 3% improvement via the temperature optimization, compared with 17% improvement via the entropy regularization. We conclude that the effect of entropy regularization is much more refined and powerful than just flattening the probability curve.

Table 4. The effect of changing the temperature of an (unregularized) GNN predictor trained for 10 iterations on the M2k dataset on the number of problems solved.

Model	$T = 0.5$	$T = 1$	$T = 2$	$T = 3$	$T = 4$	$T = 5$
GNN	1036	1050	1057	1066	1068	1061

Table 5. Number of problems solved by the GNN trained for 10 iterations on the M2k dataset with different softmax temperatures.

Iteration	0	1	2	3	4	5	6	7	8	9	10
GNN $T = 1$	523	746	850	899	938	971	992	1012	1021	1023	1050
GNN $T = 4$	523	705	800	864	894	931	993	1017	1049	1065	1079

5.6 Experiment 5: Final Large Train/Test Evaluation on Mizar40

Finally, we perform a large evaluation of **plCoP** using XGBoost and GNN on the full Mizar40 dataset, and we compare its performance with **rlCoP** and **graphCoP**. This evaluation, including the training of the GNNs on the growing sets of proofs generated from the successive proving/learning iterations, takes over 10 days on a large multicore server, and the number of training policy/value examples extracted from the MCTS proof searches goes over 5M in the last iteration.

The 32524 problems are randomly split using a 9:1 ratio into 29272 training problems and 3252 evaluation problems. For consistency, we employ the same split that was used in [25]. Successive predictors are only trained on data extracted from the training problems, but we always evaluate the predictors on both the training and evaluation sets and report the number of proofs found for them. Our results can be found in Table 6, with **plCoP**/GNN solving in the last iteration 16906 training problems and 1767 evaluation problems. These are the highest numbers obtained so far with any learning-guided **leanCoP**-based system.

For training the GNN policy predictor, we use the entropy regularization coefficient ($\alpha = 0.7$) that worked best on M2k, without its further tuning on this larger dataset. Note that the resource limits are higher in Table 6 for **rlCoP** and also a bit different for **graphCoP** (which was run only for a few iterations), as we took their published results from [25, 32] rather than rerunning the systems. Also, the evaluation of **plCoP** with GNN was stopped after iteration 8 due to our resource limits and the clear flattening of the performance on the evaluation set (1767 vs 1758 in the 8th vs 7th iteration).

Table 6. Comparing **plCoP** with XGBoost, **plCoP** with GNN, **rlCoP** and **graphCoP** on the Mizar40 training and evaluation set. **rlCoP** employs 200000 inference limit and 2000 bigstep frequency, **plCoP** uses 20000 inference limit and 200 bigstep frequency, **graphCoP** uses 200 depth limit and 200 bigstep frequency.

Iteration	0	1	2	3	4	5	6	7	8	9	10
Train set											
rlCoP	7348	12325	13749	14155	14363	14403	14431	14342	14498	14481	14487
graphCoP	4595	11978	12648	12642							
plCoP XGB	4904	8917	10600	11221	11536	11627	11938	11999	12085	12063	12151
plCoP GNN	4888	8704	12630	14566	15449	16002	16467	16745	16906		
Eval set											
rlCoP	804	1354	1519	1566	1595	1624	1586	1582	1591	1577	1621
graphCoP	510	1322	1394	1360							
plCoP XGB	554	947	1124	1158	1177	1204	1217	1210	1212	1213	1204
plCoP GNN	554	969	1375	1611	1650	1730	1742	1758	1767		

In particular, **plCoP** was given 20000 inferences per problem, i.e., one tenth of the inference limit used for **rlCoP** in [25]. For a fair comparison with **rlCoP**, we thus take the predictors (both XGBoost and GNN) used in the last **plCoP**

iteration (iteration 10 for XGBoost and iteration 8 for GNN) and run **pCoP** with them on the evaluation set with 200000 inference limit and 2000 bigstep frequency, which corresponds to the limits used for **rlCoP** in [25]. To ensure that the system has enough time to exhaust its inference limit, we increase the timeout to 6000 s. **pCoP** with XGBoost then solves 1499 of the evaluation problems while **pCoP** with GNN solves **1907** (58.6%) of them. This is our final evaluation result, which is **17.4%** higher than the 1624 evaluation problems solved by **rlCoP** in the best previously published result so far [25].

6 Conclusion

We have extended the **pCoP** learning-based connection prover with a fast, logic-aware graph neural network (GNN) and explored how the GNN can learn good guidance for selecting inferences in this setting. We have identified the entropy of the inference selection predictor as a key driver of the ATP performance and shown that Maximum Entropy Reinforcement Learning largely improves the performance of the trained policy network, outperforming simpler temperature-based entropy increasing methods. To the best of our knowledge, this is the first time that the role of entropy in guiding a theorem prover has been analyzed.

We have discovered that replacing the particular trained predictors by arbitrary (random) but entropy-normalized predictors that respect the inference ordering yields only slightly weaker theorem proving performance than the best methods. This suggests that the right inference ordering plus the right amount of entropy capture most of the benefits of the learned inference guidance. In the large final train/test evaluation on the full Mizar40 benchmark our system improves by 17.4% over the best previously published result achieved by **rlCoP**.

Acknowledgments. ZZ was supported by the European Union, co-financed by the European Social Fund (EFOP-3.6.3-VEKOP-16-2017-00002), the Hungarian National Excellence Grant 2018-1.2.1-NKP-00008 and by the Hungarian Ministry of Innovation and Technology NRDI Office within the framework of the Artificial Intelligence National Laboratory Program. JU was funded by the *AI4REASON* ERC Consolidator grant nr. 649043 and the European Regional Development Fund under the Czech project AI&Reasoning CZ.02.1.01/0.0/0.0/15_003/0000466. MO was supported by the ERC starting grant no. 714034 SMART. We thank the TABLEAUX'21 reviewers for their thoughtful reviews and comments.

References

1. Alama, J., Heskes, T., Kühlwein, D., Tsvitsivadze, E., Urban, J.: Premise selection for mathematics by corpus analysis and kernel methods. *J. Autom. Reason.* **52**(2), 191–213 (2014)
2. Alemi, A.A., Chollet, F., Een, N., Irving, G., Szegedy, C., Urban, J.: DeepMath - deep sequence models for premise selection. In: Proceedings of the 30th International Conference on Neural Information Processing Systems, NIPS 2016, USA, pp. 2243–2251. Curran Associates Inc. (2016)

3. Anthony, T., Tian, Z., Barber, D.: Thinking fast and slow with deep learning and tree search. CoRR, abs/1705.08439 (2017)
4. Bansal, K., Loos, S.M., Rabe, M.N., Szegedy, C., Wilcox, S.: HOList: an environment for machine learning of higher order logic theorem proving. In: Chaudhuri, K., Salakhutdinov, R. (eds.) International Conference on Machine Learning, ICML 2019. Proceedings of Machine Learning Research, vol. 97, pp. 454–463. PMLR (2019)
5. Bibel, W.: A vision for automated deduction rooted in the connection method. In: Schmidt, R.A., Nalon, C. (eds.) TABLEAUX 2017. LNCS (LNAI), vol. 10501, pp. 3–21. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-66902-1_1
6. Blaauwbroek, L., Urban, J., Geuvers, H.: Tactic learning and proving for the Coq proof assistant. In: Albert, E., Kovács, L. (eds.) LPAR 2020: 23rd International Conference on Logic for Programming, Artificial Intelligence and Reasoning. EPiC Series in Computing, vol. 73, pp. 138–150. EasyChair (2020)
7. Browne, C., et al.: A survey of Monte Carlo tree search methods. IEEE Trans. Comput. Intell. AI Games **4**, 1–43 (2012)
8. Chen, T., Guestrin, C.: XGBoost: a scalable tree boosting system. In: Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2016, New York, NY, USA, pp. 785–794. ACM (2016)
9. Chvalovský, K., Jakubův, J., Suda, M., Urban, J.: ENIGMA-NG: efficient neural and gradient-boosted inference guidance for E. In: Fontaine, P. (ed.) CADE 2019. LNCS (LNAI), vol. 11716, pp. 197–215. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-29436-6_12
10. The Coq Proof Assistant. <http://coq.inria.fr>
11. Crouse, M., et al.: A deep reinforcement learning approach to first-order logic theorem proving. Artificial Intelligence (2019). arXiv
12. Färber, M., Kaliszyk, C., Urban, J.: Machine learning guidance for connection tableaux. J. Autom. Reason. **65**(2), 287–320 (2021)
13. Gauthier, T., Kaliszyk, C., Urban, J.: TacticToe: learning to reason with HOL4 tactics. In: Eiter, T., Sands, D. (eds.) LPAR-21. 21st International Conference on Logic for Programming, Artificial Intelligence and Reasoning. EPiC Series in Computing, vol. 46, pp. 125–143. EasyChair (2017)
14. Gauthier, T., Kaliszyk, C., Urban, J., Kumar, R., Norrish, M.: TacticToe: learning to prove with tactics. J. Autom. Reason. **65**(2), 257–286 (2021)
15. Gordon, M.J.C., Melham, T.F. (eds.): Introduction to HOL: A Theorem Proving Environment for Higher Order Logic. Cambridge University Press, Cambridge (1993)
16. Harrison, J.: HOL light: a tutorial introduction. In: Srivas, M., Camilleri, A. (eds.) FMCAD 1996. LNCS, vol. 1166, pp. 265–269. Springer, Heidelberg (1996). <https://doi.org/10.1007/BFb0031814>
17. Huang, D., Dhariwal, P., Song, D., Sutskever, I.: Gamepad: a learning environment for theorem proving. In: 7th International Conference on Learning Representations, ICLR 2019. OpenReview.net (2019)
18. Jakubův, J., Chvalovský, K., Olšák, M., Piotrowski, B., Suda, M., Urban, J.: ENIGMA anonymous: symbol-independent inference guiding machine (system description). In: Peltier, N., Sofronie-Stokkermans, V. (eds.) IJCAR 2020. LNCS (LNAI), vol. 12167, pp. 448–463. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-51054-1_29

19. Jakubův, J., Urban, J.: ENIGMA: efficient learning-based inference guiding machine. In: Geuvers, H., England, M., Hasan, O., Rabe, F., Teschke, O. (eds.) CICM 2017. LNCS (LNAI), vol. 10383, pp. 292–302. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-62075-6_20
20. Jakubův, J., Urban, J.: Hammering Mizar by learning clause guidance. In: Harrison, J., O’Leary, J., Tolmach, A. (eds.) 10th International Conference on Interactive Theorem Proving, ITP 2019, Portland, OR, USA, 9–12 September 2019. LIPIcs, vol. 141, pp. 34:1–34:8. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2019)
21. Kaliszzyk, C., Urban, J.: M2K dataset. https://github.com/JUrban/deepmath/blob/master/M2k_list
22. Kaliszzyk, C., Urban, J.: MizAR40 dataset. <https://github.com/JUrban/deepmath>
23. Kaliszzyk, C., Urban, J.: FEMaLeCoP: fairly efficient machine learning connection prover. In: Davis, M., Fehnker, A., McIver, A., Voronkov, A. (eds.) LPAR 2015. LNCS, vol. 9450, pp. 88–96. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48899-7_7
24. Kaliszzyk, C., Urban, J.: MizAR 40 for MizAR 40. *J. Autom. Reason.* **55**(3), 245–256 (2015). <https://doi.org/10.1007/s10817-015-9330-8>
25. Kaliszzyk, C., Urban, J., Michalewski, H., Olsák, M.: Reinforcement learning of theorem proving. In: NeurIPS, pp. 8836–8847 (2018)
26. Kaliszzyk, C., Urban, J., Vyskočil, J.: Efficient semantic features for automated reasoning over large theories. In: Yang, Q., Wooldridge, M. (eds.) IJCAI 2015, pp. 3084–3090. AAAI Press (2015)
27. Kocsis, L., Szepesvári, C.: Bandit based Monte-Carlo planning. In: Fürnkranz, J., Scheffer, T., Spiliopoulou, M. (eds.) ECML 2006. LNCS (LNAI), vol. 4212, pp. 282–293. Springer, Heidelberg (2006). https://doi.org/10.1007/11871842_29
28. Kovács, L., Voronkov, A.: First-order theorem proving and VAMPIRE. In: Sharygina, N., Veith, H. (eds.) CAV 2013. LNCS, vol. 8044, pp. 1–35. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39799-8_1
29. Loos, S.M., Irving, G., Szegedy, C., Kaliszzyk, C.: Deep network guided proof search. In: 21st International Conference on Logic for Programming, Artificial Intelligence, and Reasoning (LPAR) (2017)
30. Mohamed, O.A., Muñoz, C., Tahar, S. (eds.): TPHOLs 2008. LNCS, vol. 5170. Springer, Heidelberg (2008). <https://doi.org/10.1007/978-3-540-71067-7>
31. Nagashima, Y., He, Y.: PaMpeR: proof method recommendation system for Isabelle/HOL. In: Huchard, M., Kästner, C., Fraser, G. (eds.) Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, 3–7 September 2018, pp. 362–372. ACM (2018)
32. Olsák, M., Kaliszzyk, C., Urban, J.: Property invariant embedding for automated reasoning. In: Giacomo, G.D., et al. (eds.) ECAI 2020–24th European Conference on Artificial Intelligence, Santiago de Compostela, Spain, 29 August–8 September 2020 - Including 10th Conference on Prestigious Applications of Artificial Intelligence (PAIS 2020). Frontiers in Artificial Intelligence and Applications, vol. 325, pp. 1395–1402. IOS Press (2020)
33. Otten, J.: leanCoP 2.0 and ileanCoP 1.2: high performance lean theorem proving in classical and intuitionistic logic (system descriptions). In: Armando, A., Baumgartner, P., Dowek, G. (eds.) IJCAR 2008. LNCS (LNAI), vol. 5195, pp. 283–291. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-71070-7_23
34. Otten, J., Bibel, W.: leanCoP: lean connection-based theorem proving. *J. Symb. Comput.* **36**, 139–161 (2003)
35. Rawson, M., Reger, G.: Automated theorem proving, fast and slow. EasyChair Preprint no. 4433. EasyChair (2020)

36. Ross, S., Gordon, G., Bagnell, D.: A reduction of imitation learning and structured prediction to no-regret online learning. In: Gordon, G., Dunson, D., Dudík, M. (eds.) *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics. Proceedings of Machine Learning Research*, Fort Lauderdale, FL, USA, 11–13 April 2011, vol. 15, pp. 627–635. PMLR (2011)
37. Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G.: The graph neural network model. *Trans. Neur. Netw.* **20**(1), 61–80 (2009)
38. Schulz, S.: E - a Brainiac theorem prover. *AI Commun.* **15**(2–3), 111–126 (2002)
39. Schulz, S.: System description: E 1.8. In: McMillan, K., Middeldorp, A., Voronkov, A. (eds.) *LPAR 2013. LNCS*, vol. 8312, pp. 735–743. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-45221-5_49
40. Silver, D., et al.: Mastering the game of go without human knowledge. *Nature* **550**, 354 (2017)
41. Slind, K., Norrish, M.: A brief overview of HOL4. In: Mohamed et al. [30], pp. 28–32
42. Suda, M.: New techniques that improve Enigma-style clause selection guidance. In: *International Conference on Automated Deduction, CADE 2021* (2021)
43. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*, 2nd edn. The MIT Press, Cambridge (2018)
44. Urban, J.: ERC project AI4Reason final scientific report (2021). http://grid01.ciirc.cvut.cz/~mptp/ai4reason/PR_CORE_SCIENTIFIC_4.pdf
45. Urban, J., Sutcliffe, G., Pudlák, P., Vyskočil, J.: MaLAREa SG1 - machine learner for automated reasoning with semantic guidance. In: *IJCAR*, pp. 441–456 (2008)
46. Urban, J., Vyskočil, J., Štěpánek, P.: MaLeCoP machine learning connection prover. In: Brünnler, K., Metcalfe, G. (eds.) *TABLEAUX 2011. LNCS (LNAI)*, vol. 6793, pp. 263–277. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22119-4_21
47. Vulkan, N.: An economist’s perspective on probability matching. *J. Econ. Surv.* **14**(1), 101–118 (2000)
48. Wenzel, M., Paulson, L.C., Nipkow, T.: The Isabelle framework. In: Mohamed et al. [30], pp. 33–38
49. Williams, R.J., Peng, J.: Function optimization using connectionist reinforcement learning algorithms. *Connect. Sci.* **3**(3), 241–268 (1991)
50. Yang, K., Deng, J.: Learning to prove theorems via interacting with proof assistants. In: Chaudhuri, K., Salakhutdinov, R. (eds.) *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, Long Beach, California, USA, 9–15 June 2019. Proceedings of Machine Learning Research*, vol. 97, pp. 6984–6994. PMLR (2019)
51. Zombori, Z., Urban, J., Brown, C.E.: Prolog technology reinforcement learning prover. In: Peltier, N., Sofronie-Stokkermans, V. (eds.) *IJCAR 2020. LNCS (LNAI)*, vol. 12167, pp. 489–507. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-51054-1_33